

Math-TutorK: An AI-Powered Interactive Math Tutor

Final Project Report

Course: AIE314 AI-Based Programming
University: Alamein International University

Project Team:

Mariam Sobhy (22100778)
Ramez Ezzat (22100506)
Mahmoud Eid (22100680)
Abdelrahman Harb (22100692)
Ahmed Ehab (21100778)

1 Executive Summary

Math-TutorK is an innovative, AI-powered interactive math tutoring platform designed specifically for children. By combining real-time voice interactions, dynamic visual feedback, and an advanced Large Language Model (LLM) backend, Math-TutorK emulates the experience of a personalized human tutor. The system leverages local and cloud-based AI services to guide students through elementary mathematical concepts using a step-by-step pedagogical approach, significantly improving engagement and concept retention without simply “giving away” the answers.

2 Introduction & Problem Statement

Teaching mathematics to young learners has historically relied on traditional methods that often fail to hold children’s attention or adapt to their unique learning paces. A survey conducted as part of this project, which gathered 148 responses (predominantly from students), highlighted the critical need for a modern solution:

- **High Difficulty Barrier:** 66.2% of respondents rated math learning for children as either “Moderate” (41.4%) or “Difficult” (24.8%).
- **Primary Struggles:** The primary challenges identified were understanding core concepts (28.5%), staying focused and engaged (23.6%), and solving problems step-by-step (20.6%).

Math-TutorK was developed to directly address these pain points. By incorporating a highly interactive voice interface and visually engaging lessons, the system tackles the lack of focus. Furthermore, by utilizing a fine-tuned LLM wrapped in strict educational guardrails, the tutor focuses on conceptual understanding and step-by-step problem-solving rather than rote memorization.

3 System Architecture

The Math-TutorK platform is built on a modern, decoupled client-server architecture designed for low latency and high interactivity.

3.1 Frontend (Client)

The frontend is a single-page application (SPA) built using **React** and **TypeScript**, bundled with **Vite**.

- **Styling & Animation:** **Tailwind CSS** provides utility-first styling for a vibrant, child-friendly interface, while **Framer Motion** drives smooth, engaging animations (such as confetti drops and dynamic number lines).
- **State Management:** The application uses localized state management to isolate complex lesson logic, ensuring high performance during real-time interactions.

3.2 Backend (Agent Server)

The backend is a high-performance **FastAPI** application running on Python.

- **Real-Time Communication:** The server communicates with the frontend exclusively via **WebSockets**, allowing for full-duplex, low-latency streaming of audio chunks and visual commands.
- **Tunneling:** To securely expose the local FastAPI server to the external frontend application, **Cloudflare Tunnels** (cloudflared) are utilized.

4 Core AI & Interactive Features

4.1 Advanced Voice Pipeline

A robust voice interface is critical for child-friendly applications. Our survey indicated that **69.7%** of users believe voice interaction makes learning more engaging. To achieve this, we developed a highly optimized, dual-way voice pipeline:

- **Speech-to-Text (STT) & VAD:** We implemented a Custom Adaptive NumPy Voice Activity Detection (VAD) algorithm. Unlike static threshold VADs, our adaptive system calculates dynamic background noise floors using RMS (Root Mean Square) energy, preventing microphone variations from clipping speech. A Pre-Speech Dequeue Buffer was introduced to capture the crucial 300ms of audio before a voice threshold is crossed, completely eliminating the “first-word clipping” phenomenon.
- **Text-to-Speech (TTS):** The system utilizes a dedicated local Pocket TTS server to generate warm, natural-sounding audio responses instantaneously, keeping the conversational flow intact.

4.2 LLM Intelligence & Guardrails

The cognitive engine of Math-TutorK relies on Large Language Models to simulate a real teacher.

- **Adaptive Routing:** The system defaults to high-speed cloud providers (e.g., Groq) but seamlessly falls back to a locally hosted **Llama 3.2** model via Ollama. This ensures the application remains highly available and functional even offline.
- **Pedagogical Guardrails:** To ensure child safety and maintain focus, the LLM is constrained by a strict `LLM_GUARDRAIL_SYSTEM` prompt. The system is instructed to act exclusively as a math tutor. Off-topic prompts (e.g., asking about video games or generating code) are automatically deflected, steering the conversation back to the math problem on the screen.

4.3 Interactive UI & Visual Protocol

Unlike standard chatbots, Math-TutorK manipulates the screen based on the LLM’s spatial awareness.

- **Action Tags:** The LLM generates specific Markdown-style tags (e.g., `<HIGHLIGHT>`, `<SHOW_HINT>`, `<CELEBRATE>`) at the end of its text output.
- **Visual Command Execution:** The backend parses these tags and sends WebSocket visual commands to the React frontend. The frontend dynamically renders annotations, reveals hints, or triggers Framer Motion confetti animations, creating a deeply immersive multimedia learning loop.

5 Evaluation & User Feedback

5.1 User Survey Analysis

To validate the system’s core features, we deployed a user evaluation survey and received 148 responses. The feedback overwhelmingly confirmed the product-market fit and the efficacy of our architectural decisions:

- **Interactive Validation:** **82.7%** of users agreed that real-time feedback improves learning.
- **Voice Preference:** The integration of voice technology was heavily validated, with almost 70% stating it makes learning more engaging.
- **Overall Reception:** An impressive **84.1%** of respondents stated they would recommend Math-TutorK for children learning math, and over 80% confirmed the system is engaging and child-friendly.
- **Feature Highlights:** Users identified AI Tutor Explanations (25.0%) and Voice Interaction (22.9%) as the most interesting features of the application.

5.2 LLM Evaluation Metrics

We designed an automated evaluation script to test the backend model (Llama 3.2 3B) against rigorous educational metrics, ensuring it behaves safely and effectively as a tutor.

- **Context Relevance (1.000):** Tested against math-specific prompts, the model successfully utilized math-aware vocabulary and referenced the on-screen numbers in 100% of the cases.
- **Topic Adherence (1.000):** Tested against a battery of off-topic prompts (e.g., asking for stories, code generation, or jokes), the model successfully deflected the distraction and redirected the user to the math problem 100% of the time without suffering from prompt-injection or hallucination.
- **Conciseness (0.800):** We instituted a strict 1-3 sentence rule to prevent the LLM from overwhelming the child. The model maintained this brevity in 80% of interactions, only exceeding it when providing highly detailed, step-by-step conceptual breakdowns of division.

6 Technical Challenges & Solutions

Developing a real-time, voice-activated AI tutor presented several unique engineering challenges:

1. **Background Noise and Premature Cutoffs:** The initial static threshold VAD resulted in “first-word clipping” and cut children off mid-sentence if they paused. We solved this by developing a Custom Adaptive NumPy VAD with a rolling RMS noise floor and a deque buffer that continuously captures the 300ms of audio preceding the voice activation threshold.
2. **LLM Hallucinations on Edge Hardware:** Early iterations using tiny local models (e.g., Qwen 0.8b) suffered from severe hallucinations and failed to follow the complex pedagogical guardrails. We resolved this by upgrading the local fallback to **Llama 3.2 (3B)**, which completely eliminated off-topic hallucinations while remaining fast enough to run locally.

7 Conclusion & Future Work

Math-TutorK successfully bridges the gap between static digital worksheets and personalized human tutoring. By leveraging a highly interactive React frontend, a low-latency WebSocket backend, and a carefully constrained LLM intelligence layer, the platform delivers an engaging, child-friendly educational experience.

Future work will focus on expanding the library of visual teaching playbooks, introducing adaptive difficulty scaling based on the captured telemetry data, and further optimizing the STT engine for regional accents.